Combinatorial Maximum Flow in $\tilde{O}(n^2)$ Time

Sanghoon Park

Seoul National University

Feb 24, 2026

Maximum Flow Problem

Definition

Let $G = (V, E, \mathbf{c})$ be a directed graph with a source $s \in V$, a sink $t \in V$, and a capacity vector $\mathbf{c} \in \mathbb{R}^E_{\geq 0}$. A *flow* is a vector $\mathbf{f} \in \mathbb{R}^E_{\geq 0}$ that satisfies the following conditions:

- Capacity constraint: $\mathbf{f} \leq \mathbf{c}$
- Flow conservation:
  $\forall v \in V \setminus \{s, t\}, \sum_{(u,v) \in E} \mathbf{f}((u, v)) = \sum_{(v,u) \in E} \mathbf{f}((v, u))$

Definition

The *value of a flow* $\mathbf{f}$ is given by:

$$|\mathbf{f}| = \sum_{(s,v) \in E} \mathbf{f}((s, v)) - \sum_{(u,s) \in E} \mathbf{f}((u, s))$$

The **maximum flow problem** is to find the flow $\mathbf{f}_{\max}$ with maximum value.

Time Complexity

Approach	**Complexity**	**Reference**
Augmenting Path	$O(m \cdot \min\{m^{1/2}, n^{2/3}\})$	Goldberg-Rao, JACM 1998
Interior Point Method	$\tilde{O}(m + n^{1.5})$	van den Brand et al., STOC 2021
Interior Point Method	$O(m^{1+o(1)})$	Chen et al., FOCS 2022
Augmenting Path	$\tilde{\mathbf{O}}(\mathbf{n}^2)$	**Bernstein et al., 2025**

Table: Comparison of Maximum Flow Algorithms

Interior Point Method vs. Augmenting Path

- **Interior Point Method (IPM)**
  - Reduces the problem to solving electrical flows
  - Requires complex dynamic data structures
  - Lacks direct combinatorial interpretation
- **Augmenting Path**
  - Highly intuitive and combinatorial
  - No significant improvement for 40 years

The Gap

Can we achieve near-optimal time using purely combinatorial methods?

Main Result

Theorem (Bernstein et al., 2025)

There exists a randomized algorithm that finds a maximum (s, t)-flow in

$$O(n^2 \log^{19} n \log U)$$

time with high probability.

- The **first combinatorial algorithm** to achieve $\tilde{O}(n^2)$ time.
- Significantly simplifies their previous $O(n^{2+o(1)})$ algorithm.

[Bernstein et al., FOCS 2024]

Algorithm Overview

$\tilde{O}(n^2)$ Flow Algorithm

1. Compute expander hierarchy $\mathcal{H}$ of G.
2. Build G_A by adding shortcuts over $\mathcal{H}$.
3. Run weighted push-relabel on G_A to find an approximate flow.
4. Route the approximate flow back to G.

(Repeat $O(\log n)$ times to obtain the exact maximum flow)

- **Goal:** Understanding the structural framework

Basic Definitions

Definition

Consider a graph $G = (V, E, \mathbf{c})$, an $F \subseteq E$, and an $S \subseteq V$.

- $\delta_F(v)$ is the set of edges in F incident to v.
- $\deg_F(v) = \sum_{e \in \delta_F(v)} \mathbf{c}(e)$
- $\mathrm{vol}_F(S) = \sum_{v \in S} \deg_F(v)$

Multi-Commodity Flow

Definition

Consider a flow $\mathbf{f} \in \mathbb{R}^E_{\geq 0}$. Note that we do not care about capacity constraint and flow conservation here.

- $\mathrm{cong}(\mathbf{f}) = \max \frac{\mathbf{f}(e)}{\mathbf{c}(e)}$
- $\mathbf{f}^{\mathrm{out}}(u) = \sum_{(u,v)\in E} \mathbf{f}((u,v)) - \sum_{(v,u)\in E} \mathbf{f}((v,u))$

Definition

A *vertex-demand* is a pair $(\mathbf{\Delta}, \nabla)$ with *source* and *sink* vector $\mathbf{\Delta}, \nabla \in \mathbb{R}^V_{\geq 0}$.

- $\mathbf{\Delta}_{\mathbf{f}}(u) = \max\{0, \mathbf{\Delta}(u) - \nabla(u) - \mathbf{f}^{\mathrm{out}}(u)\}$
- $\nabla_{\mathbf{f}}(u) = \max\{0, \nabla(u) - \mathbf{\Delta}(u) + \mathbf{f}^{\mathrm{out}}(u)\}$
- $|\mathbf{f}| = \sum(\mathbf{\Delta}(u) - \mathbf{\Delta}_{\mathbf{f}}(u))$

Multi-Commodity Flow

Definition

The flow $\mathbf{f}$ *routes* $(\mathbf{\Delta}, \nabla)$ if $|\mathbf{f}| = \|\mathbf{\Delta}\|_1$ and $\nabla_{\mathbf{f}} \geq \mathbf{0}$.

Definition

The flow $\mathbf{f}$ is *feasible* if $\mathbf{\Delta}_{\mathbf{f}} \leq \mathbf{\Delta}$, $\nabla_{\mathbf{f}} \leq \nabla$, and $\mathrm{cong}(\mathbf{f}) \leq 1$.

Weighted Push-Relabel

Theorem

Consider an integral maximum flow instance $\mathcal{I} = (G, \mathbf{c}, \mathbf{\Delta}, \nabla)$*, a weight function* $\mathbf{w} \in \mathbb{N}^E$*, and a height parameter* $h \in \mathbb{N}$*. There is an algorithm* $\textsc{WeightedPushRelabel}(G, \mathbf{c}, \mathbf{\Delta}, \nabla, \mathbf{w}, h)$ *that finds a feasible integral flow* $\mathbf{f}$ *such that:*

- $\mathbf{f}$ *is a* $\frac{1}{6}$*-approximation of the optimal(not necessarily integral) flow* $\mathbf{f}^*_{\mathbf{w},h}$ *with average* $\mathbf{w}$*-length* $\frac{\mathbf{w}(\mathbf{f}^*_{\mathbf{w},h})}{|\mathbf{f}^*_{\mathbf{w},h}|} \leq h$*.*

in $O((m + n + \sum \frac{h}{\mathbf{w}(e)}) \log n)$ *time.*

Weighted Push-Relabel

Goal

Our goal is to find a good weight function $\mathbf{w}$ that satisfies the following:

- There exists an approximate max flow $\mathbf{f}$ for which all flow paths P have $\mathbf{w}(P) = \tilde{O}(n)$.
- $\sum \frac{1}{\mathbf{w}(e)} = \tilde{O}(n)$.

Example

Suppose G is a DAG with a topological order τ. If we define $\mathbf{w}(u, v) = |\tau(u) - \tau(v)|$, then $\mathbf{w}$ is a good weight function because:

- All flow paths P have $\mathbf{w}(P) \leq n$.
- $\sum \frac{1}{\mathbf{w}(e)} \leq \sum_u \sum_v \frac{1}{|\tau(u)-\tau(v)|} = O(n \log n)$.

Run $\textsc{WeightedPushRelabel}(G, \mathbf{c}, \mathbf{\Delta}, \nabla, \mathbf{w}, h)$ with $h = n$ to obtain an approximate max flow. This yields an approximate max flow algorithm for DAGs that runs in $O((m + n + \sum \frac{h}{\mathbf{w}(e)}) \log n) = \tilde{O}(n^2)$ time.

Flow Expander

Definition

A demand $(\mathbf{\Delta}, \nabla)$ with $\mathbf{\Delta}(V) \leq \nabla(V)$ *respects* $\mathbf{d} \in \mathbb{R}^V_{\geq 0}$ if $\mathbf{\Delta}(v), \nabla(v) \leq \mathbf{d}(v)$ for all $v \in V$.

Definition

An edge set $F \subseteq E$ is *ϕ-flow-expanding* in G if every $\mathbf{deg}_F$-respecting demand is routable in G via a flow $\mathbf{f}$ with congestion at most $1/\phi$.

Flow Expander vs. Cut Expander

For simplicity, assume that G is an undirected graph. The following properties hold:

- If F is ϕ-flow-expanding in G, then F is ϕ-cut-expanding in G.
- If F is ϕ-cut-expanding in G, then F is $\frac{\phi}{\log n}$-flow-expanding in G.

[Leighton-Rao, 1988]

We will assume $\phi = 1/\mathrm{polylog}(n)$. Thus, we can treat these two definitions as equivalent up to polylogarithmic factors.

Weak Expander Hierarchy

Definition

A *hierarchy* $\mathcal{H}$ is defined by a *level function* $\mathrm{level} : E \to \{1, \ldots, L\}$. E_i is defined as a set of level-i edges.

Definition

A hierarchy $\mathcal{H}$ of G is ϕ-*expanding* if, for every level $i \in \{1, \ldots, L\}$, the E_i is $[G \setminus E_{>i}]$-component-constrained ϕ-expanding in G, where $[G \setminus E_{>i}]$ refers to the set of SCCs in $G \setminus E_{>i}$.

Weak Expander Hierarchy

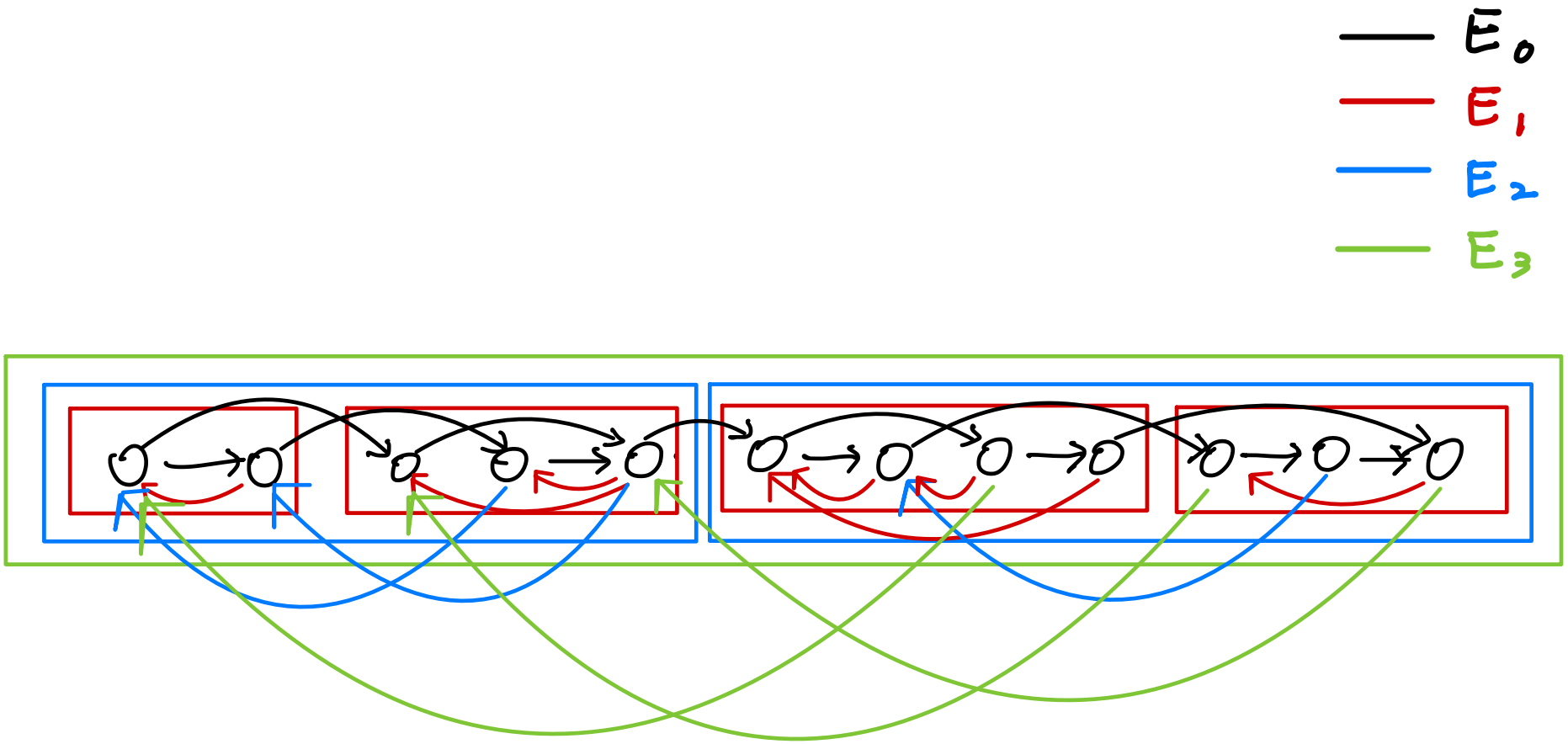
Note that the components of $\mathcal{H}$ form a laminar family. Therefore, there exists a vertex ordering $\tau_{\mathcal{H}} \in [n]^V$ that satisfies the following:

- For every SCC $C \in \mathcal{H}$, the image $\tau_{\mathcal{H}}(C)$ of C is contiguous in $[n]$.
- If u and v are in different level-i components but u can reach v in $G \setminus E_{>i}$, then $\tau_{\mathcal{H}}(u) < \tau_{\mathcal{H}}(v)$.

For simplicity, we will assume:

- E_0 is a DAG consisting of *forward* edges.
- The edges of $E_{>0}$ are *backward* edges that connect their children components.

Example



Star Edges

Definition

For any edge set $F \subseteq E$, the *star* A_F *on* F is a star whose root is a new Steiner vertex r and leaves are the tail of all edges in F.

Definition

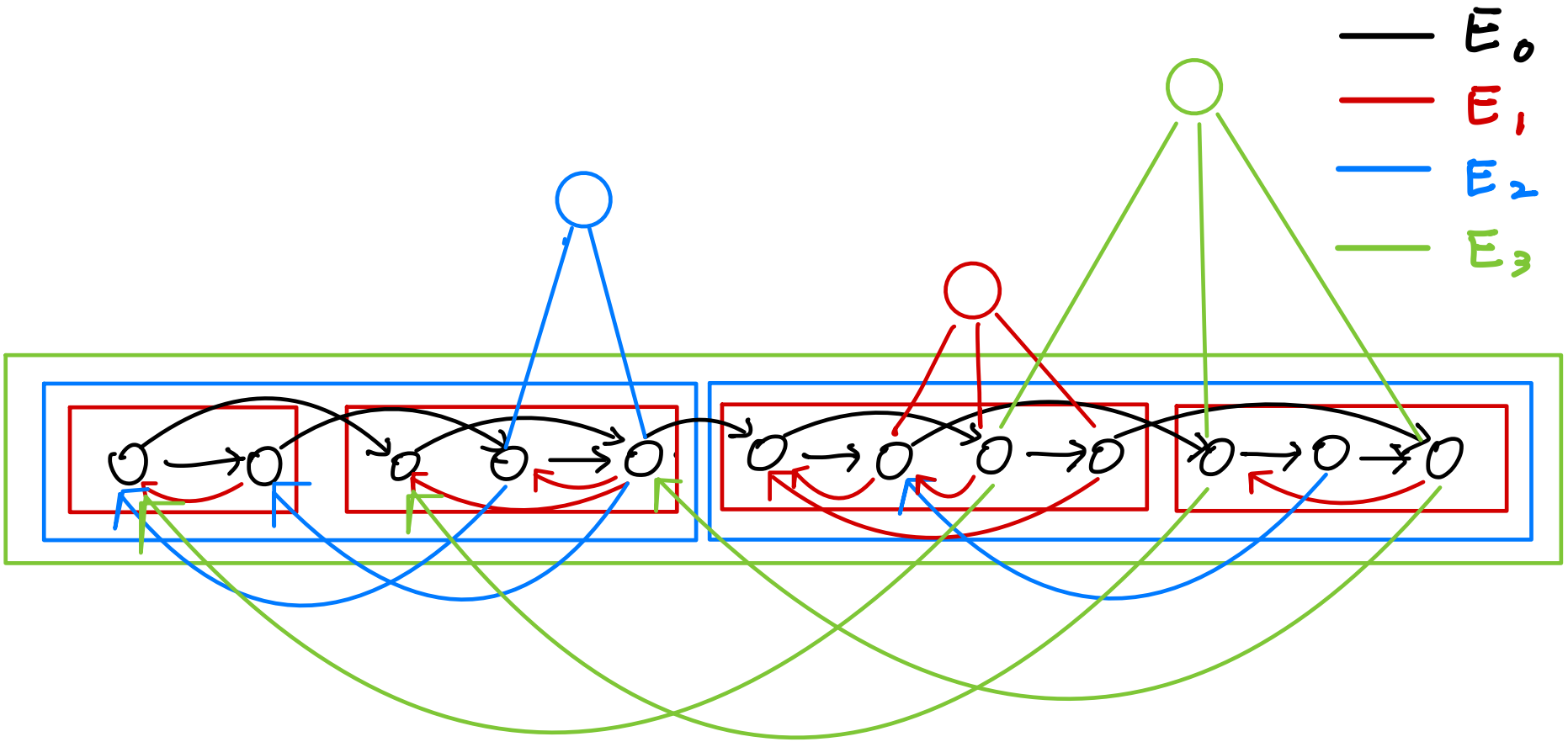
A_F has *capacity scale* $\psi \in 1/\mathbb{N}$ if the edges between the root r and the tail of $e \in F$ have capacity $\psi \cdot \mathbf{c}(e)$.

Shortcut Graph

Definition

Given a hierarchy $\mathcal{H}$ of G, the *shortcut graph* G_A is obtained by adding stars on $E_i \cap C$ for each level-i component $C \in \mathcal{C}_i$.

Example



Flow Projection

Lemma

Let F be ϕ-expanding in G. Then, adding a star A_F on F with capacity scale $\psi = \phi/\mathrm{polylog}(n)$ has minimal impact on the flow structure of G.

Weight Function

Given a shortcut graph G_A, consider the following weight function $\mathbf{w}$:

- For any original edge $e = (u, v)$, $\mathbf{w}(e) = |\tau(u) - \tau(v)|$.
- For any star edge e in component C, $\mathbf{w}(e) = |V(C)|$.

Note that the weights are large enough:

- For the original edges, $\sum 1/\mathbf{w}(e) = \tilde{O}(n)$.
- For the star edges,
  $\sum 1/\mathbf{w}(e) \leq (\#\text{ of components}) \cdot |V(H)| \cdot \frac{1}{|V(H)|} = O(n)$

Short Flow

Consider the max flow $\mathbf{f}_{\max}$ of the original graph G. We will obtain an approximate max flow $\mathbf{f}$ of the shortcut graph G_A by repeatedly *leaking* long flow paths.

Consider any level-i component C and any flow path P with one unit of flow. Let $e_1, \ldots, e_q$ be the edges of P in $C \cap E_i$.

- If $q \leq 2/\psi$, we are done.
- Otherwise, leak the flow from $\{e_1, \ldots, e_{1/\psi}\}$ to $\{e_{q-1/\psi+1}, \ldots, e_q\}$ using the star edges.

Short Flow

Observe that $\mathrm{cong}(\mathbf{f}) \leq 1$:

- For the original edges, the flow only decreases during the leaking operation.
- For the star edges, the flow is at most $\psi \cdot \mathbf{f}_{\mathsf{max}}(e) \leq \psi \cdot \mathbf{c}(e)$.

Short Flow

For any flow path P of $\mathbf{f}$, $\mathbf{w}(P) = \tilde{O}(n)$:

- Star edges: $2\sum |V(H)| = 2nL = \tilde{O}(n)$
- Backward edges: $\sum |V(H)| \cdot \frac{2}{\psi} = \tilde{O}(n)$
- Forward edges: $n +$ (total length of star edges and backward edges)

Algorithm Overview

$\tilde{O}(n^2)$ Flow Algorithm

1. Compute expander hierarchy $\mathcal{H}$ of G.
2. Build G_A by adding shortcuts over $\mathcal{H}$.
3. Run weighted push-relabel on G_A to find an approximate flow.
4. Route the approximate flow back to G.

(Repeat $O(\log n)$ times to obtain the exact maximum flow)

References

- A. Bernstein et al. "Combinatorial maximum flow via weighted push-relabel on shortcut graphs." *arXiv preprint arXiv:2510.17182*, 2025.
- A. V. Goldberg and S. Rao. "Beyond the flow decomposition barrier." *JACM*, 1998.
- J. van den Brand et al. "Minimum cost flows, mdps, and ℓ_1-regression in nearly linear time for dense instances." *STOC*, 2021.
- L. Chen et al. "Maximum flow and minimum-cost flow in almost-linear time." *FOCS*, 2022.
- A. Bernstein et al. "Maximum flow by augmenting paths in $n^{2+o(1)}$ time." *FOCS*, 2024.
- T. Leighton and S. Rao. "Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms." *JACM*, 1999.